RESIDÊNCIA TECNOLÓGICA · MÓDULO 1 · NÚCLEO COMUM

Seldon Fleet

Especificação técnica do desafio

Construir um app Android em **Kotlin/Compose** de duas telas que lê um **frota.json** local, lista os dispositivos classificando cada um como **OK**, **Atenção** ou **Crítico** por uma regra de negócio isolada em **domain/** e testável na JVM, abre o detalhe com a justificativa da classificação e persiste os favoritos — tratando os dados defeituosos do arquivo e documentando as decisões.

KOTLIN | JETPACK COMPOSE | CAMADAS | TESTES JVM | INTEGRAÇÃO CONTÍNUA | SEM REDE

Documento normativo do desafio

Versão 1.0 — revisão A

10 requisitos · 100 pontos

3 trilhas opcionais

00 Sumário

01 Contexto e propósito — por que este desafio existe e o que ele mede

02 O desafio — enunciado, escopo funcional e fronteiras

03 Especificação de dados — estrutura do frota.json e os defeitos presentes

04 Regra de classificação — derivadas, limiares, estados e liberdade de projeto

05 Requisitos e critérios de aceitação — REQ-01 a REQ-10

06 Restrições arquiteturais — camadas, dependências e inversão

07 Qualidade — testes obrigatórios e integração contínua

08 Entregáveis — o que compõe a entrega

09 Rubrica de avaliação — distribuição dos 100 pontos

10 Trilhas de especialização — extensões opcionais

11 Política de uso de IA — permitido, com declaração

12 Anexos — exemplo de dados, estrutura de pastas, glossário

01 Contexto e propósito

O que este desafio mede, e por que foi desenhado assim.

Este desafio é o instrumento de nivelamento do **Núcleo Comum** do programa. Ele não tenta cobrir as 210 horas do módulo — cobre o eixo que sustenta todos os outros: *escrever uma aplicação Android em Kotlin cuja regra de negócio sobrevive ao contato com dados reais, com testes que a prendem e documentação que explica as decisões.*

A escolha de escopo é deliberada. Um exercício de CRUD mede digitação de *boilerplate*. Um exercício com dados limpos mede quem sabe chamar uma biblioteca. Nenhum dos dois distingue um engenheiro de um copiador de exemplo. Este desafio introduz uma **decisão computada** — o status de cada dispositivo — que precisa morar numa camada sem Android, ser testável em milissegundos e continuar correta diante de um arquivo de entrada defeituoso.

Mapeamento com as disciplinas do Módulo 1

CADA ELEMENTO EXIGIDO EXISTE PORQUE UMA DISCIPLINA O COBRA

ELEMENTO DO DESAFIO	DISCIPLINA	CARGA
Kotlin, Compose, navegação, persistência local	Programação Java/Kotlin e Desenvolvimento Android	42h

ELEMENTO DO DESAFIO	DISCIPLINA	CARGA
Camadas, inversão de dependência, SOLID	Arquitetura de Software	18h
Testes unitários, integração contínua, análise estática	Testes, Automação, Segurança e Integração Contínua	30h
Requisitos com critérios de aceitação, rastreabilidade	Engenharia de Software, Requisitos e Gestão do Desenvolvimento	24h
Critério declarado antes do resultado, ameaças à validade	Dados, Experimentação e Metodologia Científica	30h
Registro de decisões, documentação de evidências, ética	Metodologia de Projetos, Ética e Habilidades Profissionais	30h
Estratégia de classificação, trilha opcional de IA	Inteligência Artificial: Fundamentos, Frameworks e Aplicações	36h

O que é avaliado de fato

Não é a quantidade de telas, nem o visual, nem o número de linhas. É a **qualidade das fronteiras**: o que a sua camada de domínio sabe, o que ela se recusa a saber, e se as suas decisões sobre dados ruins estão escritas em algum lugar que não seja a sua cabeça.

02 O desafio

Um app de duas telas que monitora uma frota de dispositivos Android.

Você recebe o *dump* de telemetria de uma frota de aparelhos Android, exportado como um único arquivo `frota.json` empacotado em `assets/`. Sua aplicação lê esse arquivo, decide o estado de saúde de cada dispositivo e apresenta esse veredito de forma compreensível e justificada.

2.1 — Tela 1: Lista da frota

- Exibe **todos** os dispositivos presentes no arquivo.
- Cada linha mostra: identificador, modelo, nível de bateria e um **chip de status** visualmente distinto — `OK` `ATENÇÃO` `CRÍTICO` `SEM DADOS`.
- Oferece **filtro por status**, com indicação da contagem de cada estado.
- Permite marcar e desmarcar um dispositivo como **favorito**.

2.2 — Tela 2: Detalhe do dispositivo

- Alcançada por navegação a partir da lista; o botão voltar do sistema retorna à lista preservando o filtro aplicado.
- Mostra os atributos do dispositivo: SoC, memória, nível de API, capacidade da bateria, consumo **já normalizado**, tempo de tela, taxa de quadros perdidos, falhas recentes e último contato.
- Mostra a **justificativa textual da classificação**, citando o valor que disparou o status. Exemplo: *"Crítico: dreno de 13,4 %/h, acima do limiar de 12 %/h."*

2.3 — Persistência

Os favoritos devem sobreviver ao encerramento do aplicativo. Nenhuma outra informação precisa ser persistida: o `frota.json` é a fonte da verdade e é relido a cada abertura.

2.4 — Fora de escopo

NÃO FAÇA

Rede, autenticação, cadastro

Sem chamadas HTTP, sem login, sem tela de registro. Os dados vêm de `assets/frota.json`. Uma camada de rede não acrescenta sinal nenhum à avaliação.

NÃO FAÇA

Banco relacional, telas extras

Room, SQLite e migrações estão fora. Favoritos cabem em `DataStore` ou equivalente. Telas adicionais não somam pontos e competem com o que soma.

O escopo é pequeno de propósito. Todo esforço gasto fora dele é esforço subtraído dos critérios que a rubrica mede.

03 Especificação de dados

O arquivo `frota.json` chega como dado real chega: com defeitos.

3.1 — Estrutura

Um objeto raiz com metadados de exportação e uma lista `devices` de aproximadamente 30 registros.

```
{
  "schemaVersion": "1.0",
  "exportedAt": "2026-10-20T03:12:44Z",
  "devices": [
    {
      "id":              "DEV-0137",
      "model":           "Pixel 6a",
      "soc":             "Tensor G1",
      "tier":            "mid",
      "ramMb":           6144,
      "androidApi":      34,
      "batteryCapacityMah": 4410,
      "batteryLevelPct": 38,
      "screenOnMinutes": 214,
      "energyConsumed":  980,
      "energyUnit":      "mAh",      // pode vir "mWh"
      "nominalVoltage":  3.85,
      "jankFramesPct":   7.4,
      "crashCount7d":    2,
      "lastSeen":        "2026-10-20T02:58:10Z"
    }
  ]
}
```

3.2 — Dicionário de campos

TIPO, UNIDADE E GARANTIAS DE CADA CAMPO

CAMPO	TIPO	UNIDADE	GARANTIA
`id`	String	—	Presente e único
`model`	String	—	Presente
`soc`	String	—	Presente
`tier`	String	`low` · `mid` · `high`	Presente
`ramMb`	Int	MB	Presente, > 0
`androidApi`	Int	nível de API	Presente
`batteryCapacityMah`	Int	mAh	**Pode ser 0 ou ausente**
`batteryLevelPct`	Int	%	0–100
`screenOnMinutes`	Int	minutos	**Pode ser 0**

CAMPO	TIPO	UNIDADE	GARANTIA
`energyConsumed`	Double	**definida por energyUnit**	Presente, ≥ 0
`energyUnit`	String	`mAh` ou `mWh`	**Varia entre registros**
`nominalVoltage`	Double	V	Presente quando `energyUnit = mWh`
`jankFramesPct`	Double	%	Presente, 0–100
`crashCount7d`	Int	contagem	Presente, ≥ 0
`lastSeen`	String	ISO-8601 UTC	Presente

3.3 — Defeitos presentes no arquivo

Dois defeitos foram deixados no arquivo de propósito. Eles não são erros de enunciado — são parte do desafio, e a rubrica os pontua.

Defeito 1 — Unidade de energia inconsistente

Parte dos registros informa `energyConsumed` em **mAh**; outra parte, em **mWh**. Somar ou comparar os dois sem converter produz erro de aproximadamente **3,85×** nos registros afetados.

A conversão usa a tensão nominal: `mAh = mWh ÷ nominalVoltage`. Exemplo: `3773 mWh ÷ 3,85 V = 980 mAh`.

Precedente histórico: em 1999 a Mars Climate Orbiter foi perdida porque um módulo de software produzia impulso em libra-força·segundo e o outro consumia em newton·segundo. Unidade não é detalhe de apresentação — é cláusula de contrato entre componentes.

Defeito 2 — Registros incompletos

Pelo menos um dispositivo tem `batteryCapacityMah` igual a `0` ou ausente, e pelo menos um tem `screenOnMinutes` igual a `0`. Ambos aparecem como divisor nas derivadas da seção 4 — ou seja, o caminho ingênuo é uma divisão por zero.

Não existe resposta única aqui. Existem três respostas defensáveis — classificar como indeterminado, excluir da lista, ou falhar explicitamente na ingestão — e uma resposta errada: exibir `NaN`, `Infinity`, um número inventado, ou deixar o app encerrar. O que a rubrica exige é que a sua escolha esteja **escrita no PROTOCOLO.md** antes de ter sido conveniente.

Por que o arquivo não vem limpo

Porque um arquivo limpo não mede nada. Com dados limpos, todo candidato entrega a mesma lista bonita e o instrumento de avaliação não consegue separar ninguém. Os defeitos são o que transforma este exercício de uma prova de digitação numa prova de engenharia.

04 Regra de classificação

A regra de referência. Você pode substituí-la — desde que justifique.

4.1 — Derivadas

Duas grandezas intermediárias, calculadas **após** a normalização de unidades:

DERIVADA	FÓRMULA	UNIDADE
Dreno absoluto	`energiaMah ÷ (screenOnMinutes ÷ 60)`	mAh/h
Dreno relativo	`(drenoAbsoluto ÷ batteryCapacityMah) × 100`	%/h

O **dreno relativo** é a grandeza usada na classificação, e não o absoluto. A razão é comparabilidade: 900 mAh/h é catastrófico num aparelho de 3000 mAh e apenas ruim num de 5000 mAh. Comparar valores absolutos entre aparelhos de capacidades diferentes é um erro de normalização, e detectá-lo vale ponto.

4.2 — Limiares

AVALIADOS NESTA ORDEM; O PRIMEIRO QUE CASAR DEFINE O STATUS

STATUS	CONDIÇÃO (QUALQUER UMA BASTA)	SIGNIFICADO

`INDETERMINADO`	`capacidade ≤ 0 · ausente`
	`ou screenOnMinutes ≤ 0`
	`ou unidade mWh sem nominalVoltage`
		Dados insuficientes para decidir

`CRÍTICO`	`drenoRelativo ≥ 12 %/h`
	`ou crashCount7d ≥ 3`
	`ou jankFramesPct ≥ 15`
		Exige intervenção

`ATENÇÃO`	`drenoRelativo ≥ 7 %/h`
	`ou crashCount7d ≥ 1`
	`ou jankFramesPct ≥ 8`
		Sob observação

`OK`	`nenhuma das anteriores`	Saudável

4.3 — Estados e apresentação

O domínio trabalha com quatro estados nomeados — `OK`, `ATENCAO`, `CRITICO`, `INDETERMINADO`. A interface pode rotulá-los como quiser (o chip `SEM DADOS` para `INDETERMINADO`, por exemplo). Essa distinção é intencional: **o nome do estado pertence ao domínio; o texto exibido pertence à apresentação**. Se a sua enumeração de domínio contém a string exibida na tela, as duas camadas vazaram uma na outra.

4.4 — Liberdade de projeto

Os limiares de 4.2 são uma **referência**, não um dogma. Você pode propor outros — por percentil da própria frota, por `tier` de SoC, por combinação ponderada de sinais. A única exigência é que a regra adotada esteja declarada no `PROTOCOLO.md` **antes** dos resultados, com o raciocínio.

A ordem importa mais do que o número

Declarar o critério antes de olhar o resultado é o que separa uma medição de uma narrativa construída depois do fato. Em ciência experimental isso se chama pré-registro, e existe por um motivo: quando o critério é escolhido depois, ele é sempre escolhido para favorecer o resultado que se obteve — sem má-fé, por construção.

05 Requisitos e critérios de aceitação

Dez requisitos. Cada um com um critério verificável por terceiros.

REQ-01 Ingestão local — OBRIGATÓRIO

O app carrega os dispositivos de `assets/frota.json`, sem qualquer acesso à rede.

> **Aceitação:** com o dispositivo em modo avião e sem permissão de `INTERNET` no manifest, a lista é exibida integralmente.

REQ-02 Normalização de unidades — OBRIGATÓRIO

Valores em `mWh` são convertidos para `mAh` via `nominalVoltage` antes de qualquer comparação, agregação ou classificação.

> **Aceitação:** existe teste unitário que, dado um registro em `mWh` e outro em `mAh` representando o **mesmo** consumo físico, produz o mesmo dreno e o mesmo status.

REQ-03 Registros incompletos — OBRIGATÓRIO

Registros com divisor nulo ou campo ausente recebem tratamento explícito e documentado. Nenhum caminho pode produzir `NaN`, `Infinity`, valor arbitrário silencioso ou encerramento do app.

> **Aceitação:** teste unitário cobrindo capacidade `0` e `screenOnMinutes = 0`; a decisão adotada consta do `PROTOCOLO.md`.

REQ-04 Regra no domínio — OBRIGATÓRIO

A classificação vive em `domain/`, em Kotlin puro: sem `import android.*`, sem conhecimento de JSON, de arquivo, de `Context` ou de framework de UI.

> **Aceitação:** `grep -r "import android" domain/` não retorna nada, e os testes da regra executam com `./gradlew test` sem emulador.

REQ-05 Tela de lista — OBRIGATÓRIO

Lista todos os dispositivos com identificador, modelo, bateria e chip de status visualmente distinto. Filtro por status com contagem por estado.

> **Aceitação:** a soma das contagens por status é igual ao total de registros do arquivo; selecionar um filtro reduz a lista coerentemente.

REQ-06 Tela de detalhe com justificativa — OBRIGATÓRIO

Navegação lista → detalhe. O detalhe exibe os atributos do dispositivo e a razão da classificação citando o valor que a disparou.

> **Aceitação:** para um dispositivo crítico, o texto nomeia o sinal responsável e o valor observado; o botão voltar retorna à lista com o filtro preservado.

REQ-07 Favoritos persistentes — OBRIGATÓRIO

Marcar um dispositivo como favorito sobrevive ao encerramento completo do processo.

> **Aceitação:** marcar, encerrar o app, reabrir — a marcação permanece.

REQ-08 **Testes unitários na JVM** OBRIGATÓRIO

Testes cobrindo, no mínimo: conversão de unidade, registro incompleto, e um caso em cada fronteira de limiar (imediatamente abaixo e imediatamente acima).

> **Aceitação:** `./gradlew test` conclui verde, sem emulador, e os testes falham se os limiares forem alterados.

REQ-09 **Integração contínua** OBRIGATÓRIO

Pipeline configurado executando análise estática e a suíte de testes a cada *push*.

> **Aceitação:** a última execução registrada no repositório está verde e inclui tanto o passo de análise estática quanto o de testes.

REQ-10 **Protocolo de decisões** OBRIGATÓRIO

`PROTOCOLO.md` contendo: a regra de classificação adotada e seu raciocínio; o tratamento dado a cada defeito da seção 3.3; e ao menos uma **ameaça à validade** da sua própria conclusão.

> **Aceitação:** um avaliador que leia apenas o `PROTOCOLO.md` consegue prever o status que o app atribui a um registro dado.

Sobre a ameaça à validade

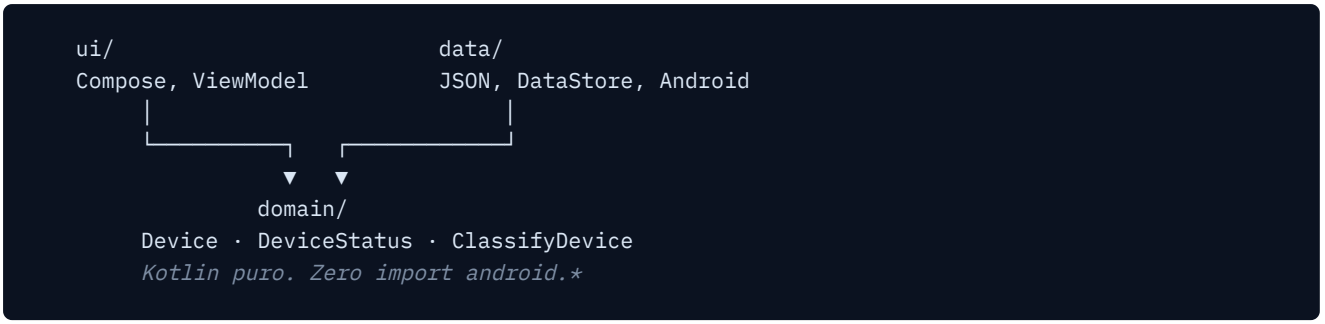
É o requisito que mais gente trata como formalidade e que mais revela maturidade. Um exemplo do que se espera: *"o dreno relativo assume que todo o consumo registrado ocorreu com a tela ligada; para aparelhos com serviço em primeiro plano e tela desligada, a métrica superestima o dreno e pode gerar falso Crítico."* Isso é conhecer o próprio instrumento.

06 Restrições arquiteturais

Três camadas e uma única regra de dependência.

6.1 — A regra

As dependências apontam para dentro. `ui` conhece `domain`. `data` conhece `domain`. E `domain` não conhece ninguém.



A inversão acontece na fronteira do domínio: `domain` declara a **interface** `DeviceRepository`; `data` a implementa lendo o arquivo. Assim o domínio depende de uma abstração que ele próprio define, e a infraestrutura depende do domínio — nunca o contrário.

6.2 — Por que isto importa, e não é burocracia

Essa separação é o que mantém o seu teste rodando em milissegundos na JVM em vez de minutos num emulador. A ideia não é nova: Alistair Cockburn formulou a *arquitetura hexagonal* em 2005 justamente porque estava cansado de suítes de teste que exigiam banco, servidor e interface de pé para verificar uma regra de três linhas. Robert Martin reempacotou o princípio como *Clean Architecture* em 2012.

A consequência prática neste desafio: a sua regra de classificação precisa ser verificável sem Android. Se para testá-la você precisa de um `Context`, ela está na camada errada — e isso é visível em dez segundos de leitura de código.

6.3 — Injeção de dependência

`ClassifyDevice` e `DeviceRepository` são **recebidos**, não instanciados dentro de quem os usa. Construtor manual é aceito; um contêiner de DI é aceito; `object` singleton acessado globalmente não é.

O teste é simples: substituir a implementação de `DeviceRepository` por uma de teste deve exigir **zero** alteração em `domain/` e **zero** alteração na regra.

6.4 — Convenções aceitas

MÓDULOS	Módulos Gradle separados ou pacotes dentro de um módulo único — ambos aceitos, desde que a regra de dependência seja respeitada e verificável.
PADRÃO DE UI	MVVM com `ViewModel` é o esperado; MVI é aceito. Lógica de classificação dentro de `Composable` não é.
SERIALIZAÇÃO	Livre: `kotlinx.serialization`, Moshi, Gson ou parsing manual.
NAVEGAÇÃO	Livre: Navigation Compose ou troca de estado. Duas telas distintas e botão voltar funcional são o requisito.

07 Qualidade

Testes que prendem comportamento e um pipeline que os executa.

7.1 — Testes obrigatórios

CASO	O QUE ELE PRENDE
Equivalência de unidade	Mesmo consumo físico em `mAh` e em `mWh` produz o mesmo status
Capacidade nula ou ausente	Resultado é o estado definido no protocolo, sem exceção e sem `NaN`
Tempo de tela igual a zero	Idem — o divisor nulo não escapa
Fronteira inferior do limiar	Valor imediatamente abaixo **não** promove o status
Fronteira superior do limiar	Valor imediatamente acima promove o status
Precedência entre sinais	Um sinal crítico vence um sinal de atenção simultâneo

> **Critério de qualidade do teste**
>
> Um teste vale se ele **falha quando a regra muda**. Teste que passa com qualquer implementação não é teste, é decoração de cobertura. O par de testes de fronteira existe exatamente por isso: ele é imune a quem "ajusta o limiar até passar".

7.2 — Integração contínua

Um *workflow* com dois passos, no mínimo: análise estática (`ktlint` , `detekt` ou `lint`) e `./gradlew test` . Deve rodar a cada *push* e estar verde na entrega.

Observação prática: o passo de testes é a JVM, não instrumentação. Não há emulador no pipeline, e por isso ele é rápido e confiável. Essa é uma consequência direta da restrição arquitetural da seção 6 — arquitetura e velocidade de CI são o mesmo assunto.

08 Entregáveis

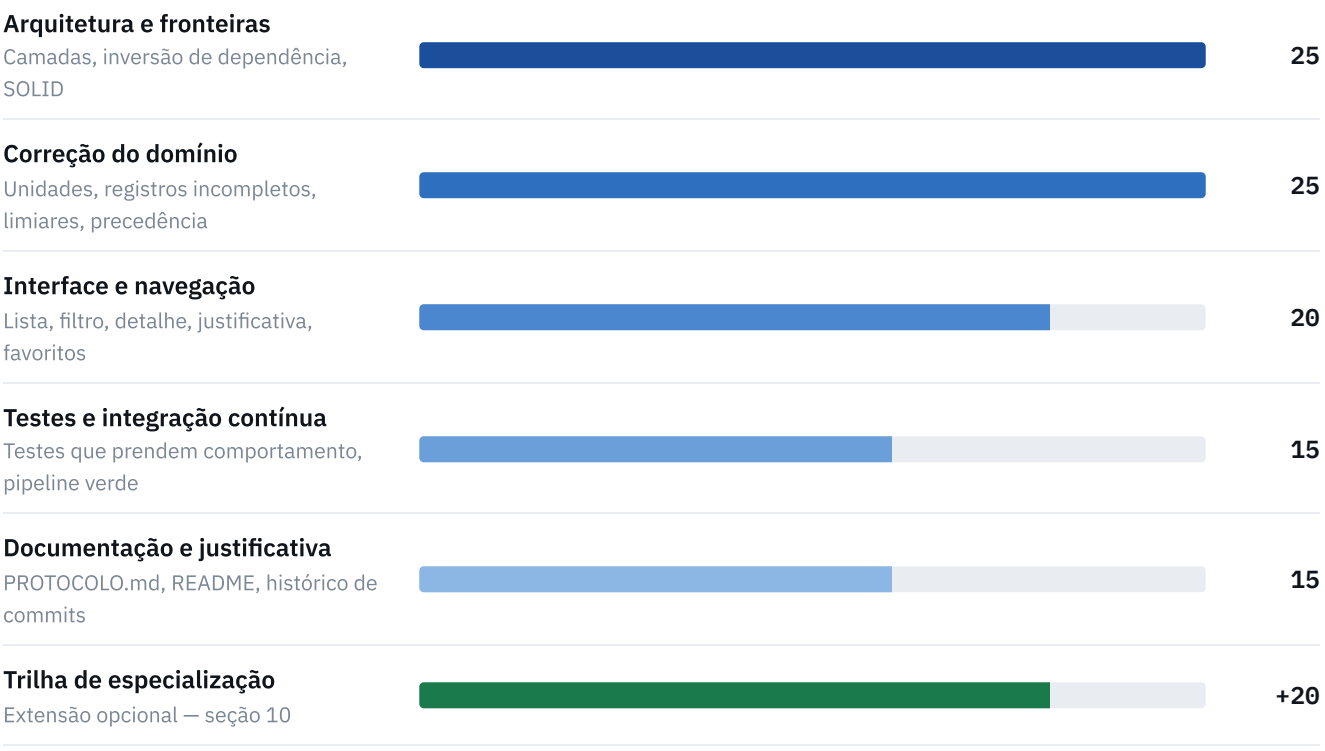
O que compõe a entrega, e nada além disso.

ARTEFATO	CONTEÚDO
`Repositório Git`	Histórico de *commits* legível — mensagens que digam o que mudou e por quê
`Código-fonte`	Projeto Android compilando, organizado conforme a seção 6
`PROTOCOLO.md`	Regra adotada, tratamento dos defeitos, ameaça à validade (REQ-10)
`README.md`	Como executar, decisões de projeto, *trade-offs* aceitos, o que faria em seguida
`CI verde`	Última execução registrada, passando (REQ-09)
`Captura de tela`	Uma imagem de cada tela, no README

> **A seção mais subestimada do README**
> "O que eu faria em seguida." Ela é lida com atenção porque mostra se você sabe onde a sua própria solução é fraca. Reconhecer uma limitação com precisão vale mais que esconder a limitação com silêncio — e é exatamente o que se espera de alguém que vai trabalhar em equipe sobre código que outros vão manter.

09 Rubrica de avaliação

Cem pontos, distribuídos pelo que o desafio se propôs a medir.



9.1 — Condições que limitam a nota

CONDIÇÃO OBSERVADA	EFEITO
`import android.*` dentro de `domain/`	Zera o eixo de arquitetura
Classificação escrita dentro de um `Composable` ou do parser	Zera o eixo de arquitetura
Unidades não normalizadas (defeito 1 não tratado)	Zera o eixo de correção do domínio
`NaN` , `Infinity` ou encerramento em registro incompleto	Zera o eixo de correção do domínio
Ausência de `PROTOCOLO.md`	Limita a nota total a 60
CI vermelha ou ausente na entrega	Zera o eixo de testes e CI
Suíte de testes vazia ou que passa com qualquer implementação	Zera o eixo de testes e CI

As condições acima não são punições arbitrárias: cada uma delas descreve um caso em que o eixo correspondente simplesmente não pode ser avaliado.

10 Trilhas de especialização

Extensões opcionais sobre o mesmo app. Escolha no máximo uma.

As trilhas existem para sinalizar afinidade, não para compor a nota mínima. Elas valem pontos adicionais e são avaliadas pela **profundidade de uma ideia**, não pela quantidade de código. Nenhuma delas substitui um requisito obrigatório: uma trilha entregue sobre um núcleo incompleto não é considerada.

Trilha A — IA aplicada à otimização

Fornecer uma **segunda implementação** da interface de classificação, orientada por dados em vez de limiares fixos — por percentil da frota, por agrupamento, ou por um modelo simples embarcado. Comparar com a regra de referência e declarar qual erra menos, em qual caso, e por quê.

O que se avalia: a comparação contra a linha de base, não a sofisticação do modelo. Uma regra de percentil bem justificada vence uma rede neural sem comparação.

Trilha B — Segurança e automação de validações

Endurecer o app e automatizar a verificação disso: revisar o `AndroidManifest` (`exported`, `debuggable`, tráfego em texto claro, *backup*), garantir que nenhum identificador de dispositivo apareça em *log*, e escrever uma verificação automatizada que falhe o *build* caso uma dessas regras seja violada.

O que se avalia: a verificação ser automatizada e o tratamento explícito de um **falso positivo** — porque um verificador que grita em tudo é um verificador que ninguém usa. Discutir o que nestes dados seria dado pessoal, sob a LGPD, soma.

Trilha C — Eficiência energética

Agendar a releitura da frota com `WorkManager` e justificar a política escolhida (frequência, restrições, comportamento em *Doze*) a partir do custo energético estimado. Responder, com os dados do arquivo: *a recomendação muda quando você separa os dispositivos por `tier` de SoC?*

O que se avalia: a justificativa quantitativa da política. A pergunta sobre `tier` tem resposta interessante no arquivo — vale procurá-la antes de responder.

11 Política de uso de IA

Permitido. Declarado.

O uso de assistentes de IA é **permitido e esperado** — seria incoerente avaliar candidatos a um programa de inteligência artificial proibindo a ferramenta. A exigência é de transparência: registre no `README.md`, em poucas linhas, onde a IA foi usada e qual parte do raciocínio é seu.

Isso não é uma armadilha moral; é a mesma disciplina de registro de decisões que a seção 8 cobra. E tem consequência prática na avaliação: a entrevista técnica parte do seu código. Quem aceitou código que não entende descobre isso na frente de outra pessoa — e é um lugar desconfortável para descobrir.

12 Anexos

Anexo A — Amostra do `frota.json` com os dois defeitos

```
"devices": [
  { // registro normal, em mAh
    "id": "DEV-0137", "model": "Pixel 6a", "tier": "mid",
    "batteryCapacityMah": 4410, "screenOnMinutes": 214,
    "energyConsumed": 980, "energyUnit": "mAh",
    "nominalVoltage": 3.85, "jankFramesPct": 7.4, "crashCount7d": 2
  },
  { // DEFEITO 1 — unidade em mWh: dividir por nominalVoltage
    "id": "DEV-0204", "model": "Galaxy A54", "tier": "mid",
    "batteryCapacityMah": 5000, "screenOnMinutes": 186,
    "energyConsumed": 3773, "energyUnit": "mWh",
    "nominalVoltage": 3.85, "jankFramesPct": 4.1, "crashCount7d": 0
  },
  { // DEFEITO 2 — capacidade 0: divisor nulo no dreno relativo
    "id": "DEV-0311", "model": "Moto G13", "tier": "low",
    "batteryCapacityMah": 0, "screenOnMinutes": 97,
    "energyConsumed": 612, "energyUnit": "mAh",
    "nominalVoltage": 3.8, "jankFramesPct": 11.2, "crashCount7d": 1
  }
]
```

Conferência dos dois primeiros: `DEV-0137` → 980 ÷ (214/60) = 274,8 mAh/h → 274,8 ÷ 4410 × 100 = **6,2 %/h**, abaixo de 7, mas `crashCount7d = 2` ≥ 1 ⇒ `ATENÇÃO`. `DEV-0204` → 3773 ÷ 3,85 = 980 mAh → 980 ÷ (186/60) = 316,1 mAh/h → **6,3 %/h**, nenhum sinal disparado ⇒ `OK`. Sem a conversão, o mesmo registro apareceria com 24,3 %/h e seria classificado `CRÍTICO`. O defeito 1 inverte o veredito — não é um arredondamento.

Anexo B — Estrutura de pastas sugerida

```
app/
├── domain/                      Kotlin puro — zero import android.*
│   ├── model/Device.kt
│   ├── model/DeviceStatus.kt        OK · ATENCAO · CRITICO · INDETERMINADO
│   ├── repository/DeviceRepository.kt interface declarada aqui
│   └── usecase/ClassifyDevice.kt    a regra
├── data/
│   ├── dto/DeviceDto.kt             espelha o JSON, inclusive os defeitos
│   ├── mapper/DeviceMapper.kt       normaliza unidades aqui
│   ├── source/AssetFleetSource.kt
│   └── repository/FleetRepositoryImpl.kt
├── ui/
│   ├── list/FleetListScreen.kt
│   ├── list/FleetListViewModel.kt
│   ├── detail/DeviceDetailScreen.kt
│   └── theme/
└── src/test/                    roda na JVM, sem emulador
    ├── ClassifyDeviceTest.kt
    └── DeviceMapperTest.kt
```

A estrutura é sugestão, não requisito. O requisito é a regra de dependência da seção 6.1 — qualquer organização que a respeite e a torne evidente é aceita.

Anexo C — Glossário

DRENO ABSOLUTO	Consumo por hora de tela ligada, em mAh/h.
DRENO RELATIVO	Dreno absoluto como percentual da capacidade da bateria, em %/h. É a grandeza comparável entre aparelhos.
JANK	Quadro que perde o prazo de composição e causa engasgo visível. Aqui, percentual de quadros perdidos na janela observada.
TIER	Faixa de desempenho do SoC: `low`, `mid`, `high`. Agrupa aparelhos comparáveis entre si.
DOZE	Modo do Android que suspende trabalho em segundo plano quando o aparelho está parado e com a tela desligada.
AMEAÇA À VALIDADE	Razão concreta pela qual a sua conclusão poderia estar errada mesmo com o código correto.